

Guia de Operações

Operação do dia a dia da plataforma

Ciclartech · Plataforma de Gestão de Ativos Assistivos

Guia de Operações

Procedimentos do dia a dia de quem opera a plataforma (equipe Ciclartech — área `/owner/`) e de manutenção do sistema em produção. Para variáveis de ambiente e deploy inicial, ver `README.md`. Para adicionar um cliente novo passo a passo, ver `docs/ONBOARDING_TENANT.md`.

1. Acesso à área Owner

`/owner/` é restrita a usuários com `is_platform_staff=True` (owner/`decorators.py::owner_required`). Um usuário assim não pertence a nenhum tenant (`tenant_id IS NULL`, reforçado por constraint de banco).

Criar o primeiro usuário Owner (não há tela para isso — é sempre via shell, deliberadamente, porque é a chave mestra da plataforma):

```
python manage.py shell
```

```
from contas.models import Usuario
Usuario.objects.create_superuser(
    username="owner_ciclartech",
    email="equipe@ciclartech.com.br",
    password="<senha forte>",
)
u = Usuario.objects.get(username="owner_ciclartech")
u.is_platform_staff = True
u.save()
```

2. Provisionar um tenant novo

Ver passo a passo completo em `docs/ONBOARDING_TENANT.md`. Resumo:

`/owner/tenants/novo/` → preencher nome/slug/segmento/cidade → criar primeiro Administrador (`/owner/tenants/<id>/administrador/`, senha temporária mostrada uma única vez) → repassar credencial ao cliente por canal seguro.

3. Ligar/desligar módulos (feature flags) de um tenant

`/owner/tenants/<id>/` lista o catálogo inteiro de `Modulo` com toggle. Um módulo tem padrão por segmento (ex.: Locadora nasce com `locacao_financeiro` e `documento_pessoa_juridica` ligados); o toggle cria um `TenantModulo` que sobrepõe esse padrão só para aquele tenant. Exige POST (`owner/views.py::alternar_modulo`), efeito imediato na próxima requisição do tenant afetado.

Para adicionar um módulo novo ao catálogo (não é operação de tela — é mudança de código/migration), ver `docs/GUIA_DESENVOLVEDOR.md` §9.

4. Suspende / reativar um contrato

`/owner/tenants/<id>/` → botão "Suspende"/"Reativar" (`alternar_tenant_ativo` , exige POST). `Tenant.ativo=False` não apaga nada — apenas o campo é marcado. Hoje ele tem um efeito concreto conhecido (o job diário de notificações só processa `Tenant.objects.filter(ativo=True)` — `notificacoes/jobs.py`), mas **não bloqueia login** automaticamente na camada de autenticação. Se for usado como controle de inadimplência para impedir acesso, tratar como item a verificar/reforçar — hoje um usuário de um tenant suspenso ainda consegue autenticar normalmente.

5. Job diário de notificações (vencimento/atraso)

Verifica empréstimos e dispara aviso 7 dias antes do vencimento, no vencimento e em atraso.

- **Local/manual:** `python manage.py enviar_notificacoes_diarias`
- **Produção (Vercel):** `vercel.json` já declara um cron (`/cron/notificacoes-diarias/` às 12:00 UTC) que chama `core/views_cron.py` , protegido por `CRON_SECRET` — sem essa env var configurada o endpoint recusa qualquer chamada.

Checar se rodou: `notificacoes_notificacaoenviada` filtrado por `criado_em` do dia, ou o painel "Notificações" de cada tenant. Em caso de falha, o comando é idempotente o bastante para rodar de novo no mesmo dia sem duplicar envio já confirmado — mas confirme `status` de cada notificação antes de reexecutar em massa.

6. Trilha de auditoria

Consultar

- Por tenant: tela "Auditoria" dentro do próprio tenant (Admin/Gestor).
- Cross-tenant (visão da plataforma): `/owner/auditoria/` (`owner/views.py::auditoria`), com filtro por tenant/ação/usuário/ "só dado sensível".
- Exportação CSV: mesmo filtro, botão de exportar (`auditoria_exportar` / `exportar_auditoria_csv`).

Expurgo por retenção (LGPD Art. 16)

```
python manage.py expurgar_auditoria          # simula (dry-run), não apaga nada
python manage.py expurgar_auditoria --confirmar  # executa de fato
```

Retenção padrão: 24 meses (`AUDITORIA_RETENCAO_DIAS` , configurável via `DJANGO_AUDITORIA_RETENCAO_DIAS`). Rodar via agendamento (cron/Vercel Cron) numa

cadência mensal ou trimestral é recomendado — hoje é comando manual, não há agendamento automático dele (diferente do job de notificações).

7. Banco de dados

Migrations

```
python manage.py showmigrations      # o que já foi aplicado
python manage.py migrate             # aplica pendentes
python manage.py migrate <app> <numero> # aplica até uma migration específica
(inclusive reverter)
```

Produção usa o mesmo comando apontando para a `DATABASE_URL` do Supabase — ver `README.md` §"Deploy na Vercel" para o procedimento completo (a Vercel não roda `migrate` automaticamente; é passo manual pós-deploy).

Novo projeto Supabase / novo ambiente de banco

Depois do primeiro `migrate` num projeto Supabase novo, **sempre** rodar:

```
psql "$DATABASE_URL" -f scripts/supabase_hardening.sql
```

Revoga privilégio de `anon / authenticated` sobre o schema `public` e ativa RLS sem policy em toda tabela — sem isso, qualquer projeto Supabase novo (staging, cópia de teste, restore de backup) nasce com as tabelas acessíveis via API REST pública do Supabase. É idempotente; rodar de novo depois de qualquer migration que crie tabela nova.

Backup e restauração

O projeto Supabase de produção tem backup gerenciado pelo próprio Supabase (frequência e retenção dependem do plano contratado — conferir no painel do projeto, seção Database → Backups). Não há rotina de backup adicional neste repositório. Antes de qualquer operação destrutiva em produção (migration que remove coluna/tabela, expurgo manual fora do comando oficial), confirmar que existe um backup recente restaurável.

Para restaurar localmente a partir de um dump:

```
pg_restore --clean --if-exists -d "$DATABASE_URL" arquivo.dump
```

Consulta ad-hoc direta no banco

Lembrar sempre: consulta via `psql` /ferramenta de BI **não passa pelo `TenantManager`** — não há isolamento automático fora do Django. Toda consulta manual precisa filtrar `tenant_id` explicitamente (ver `docs/REFERENCIA_BANCO_DE_DADOS.md` §4).

8. Storage de mídia

Arquivos de usuário (fotos de ativo, documento de beneficiário, assinatura de termo) vivem no bucket `ciclartech-media` do Supabase Storage em produção (protocolo S3), disco local em desenvolvimento. Ver `README.md` §"Storage de mídia" para as variáveis de ambiente e a diferença de tratamento entre foto de ativo (URL assinada direta) e documento de beneficiário (sempre via `core.arquivos.resposta_de_download`, autenticado).

Gerenciamento do bucket (criar, rotacionar access key S3, verificar política de privacidade) é feito direto no painel do Supabase — **Project Settings → Storage**. As access keys S3 ignoram RLS completamente; nunca reusar/expor a mesma chave em outro contexto que não seja o servidor Django.

9. Monitoramento

Não há ferramenta de monitoramento de erro (Sentry ou equivalente) integrada nesta fase — é item pendente conhecido (ver `README.md` §"Pendências de segurança"). Hoje, a única forma de descobrir uma exceção em produção é o painel de logs da Vercel (Runtime Logs do projeto) — não há alerta automático. Ao investigar um incidente reportado por usuário, comece por ali; erros `WARNING+` de `django.security` / `django.request` aparecem no mesmo stream (ver `LOGGING` em `ciclartech/settings.py`).

O log de aplicação **nunca** contém dado pessoal (nome, telefone, conteúdo de notificação) por desenho — para saber "quem acessou o quê", a fonte é sempre a trilha de auditoria, não o log.

10. Recuperação de senha / suporte a usuário final

Sem SMTP configurado (`DJANGO_EMAIL_*`), o link de redefinição só é escrito no log — não chega ao usuário. Se um cliente reportar "não recebi o e-mail de redefinição" e o SMTP está configurado corretamente, verificar: (1) pasta de spam do destinatário, (2) `PASSWORD_RESET_TIMEOUT` (padrão 1h — link pode ter expirado), (3) logs de erro do provedor SMTP.

Alternativa operacional: Admin do próprio tenant pode gerar nova senha temporária para qualquer usuário que ele gerencia, pela tela de usuários (`/app/usuarios/` , ação "Gerar nova senha") — não depende de e-mail.

11. Checklist de saúde periódica (sugerido)

Sem automação hoje — conferir manualmente em cadência definida pela operação (ex.: semanal):

- [] Job diário de notificações rodou nos últimos dias (`notificacoes_notificacaoenviada` com `criado_em` recente)

- [] Nenhum tenant com `Movimentacao` de empréstimo muito atrasada sem notificação correspondente
- [] Painel de logs da Vercel sem `ERROR / CRITICAL` recorrente
- [] Backup do Supabase disponível e dentro da janela de retenção esperada
- [] `python manage.py check --deploy --fail-level WARNING` limpo (deve estar garantido pelo CI, mas vale conferir após mudança manual de variável de ambiente)

12. Hardening pendente (conhecido, não resolvido nesta fase)

Ver lista completa e justificativa em `README.md` §"Pendências de segurança que dependem de decisão ou serviço externo": monitoramento de erro, envio de e-mail transacional, 2FA para Admin, `style-src 'unsafe-inline'` na CSP, indicação formal de Encarregado (DPO). Adicionalmente, do ponto de vista de operação de banco: a revogação do privilégio `DELETE / UPDATE` a nível de Postgres (para o usuário de aplicação) sobre as tabelas `append-only` (`ativos_movimentacao`, `ativos_impressaoetiqueta`, `auditoria_registroauditoria`) ainda não foi feita — o bloqueio hoje é só na camada de aplicação (Django model).